DAA LAB 12

Kshitij Kejriwal-252010005

1st April 2026

Bellman-Ford Algorithm

1 Code:

```python
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.edges = []

    def add_edge(self, u, v, w):
        self.edges.append((u, v, w))

    def bellman_ford(self, src):
        dist = [float('inf')] * self.V
        dist[src] = 0

        # Relax edges V-1 times
        for _ in range(self.V - 1):
            for u, v, w in self.edges:
                if dist[u] != float('inf') and dist[u] + w < dist[v]:
                    dist[v] = dist[u] + w

        # Check for negative cycle
        for u, v, w in self.edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                print("Graph contains negative weight cycle")
                return None

        return dist


# -------- USER INPUT --------
n = int(input("Enter number of vertices: "))
```

```python
m = int(input("Enter number of edges: "))

g = Graph(n)

print("Enter edges (u v weight):")
for _ in range(m):
    u, v, w = map(int, input().split())
    g.add_edge(u, v, w)

src = int(input("Enter source node: "))

# -------- OUTPUT --------
dist = g.bellman_ford(src)

if dist:
    print("\nShortest distances from source node:", src)
    for i in range(len(dist)):
        print(f"{src} -> {i} = {dist[i]}")
```

2 Output:

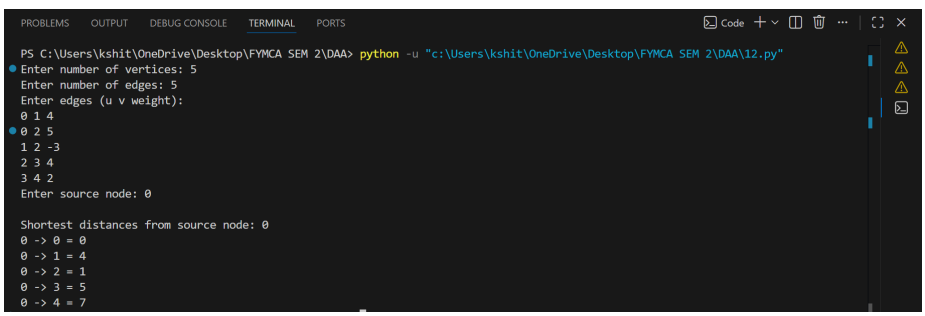


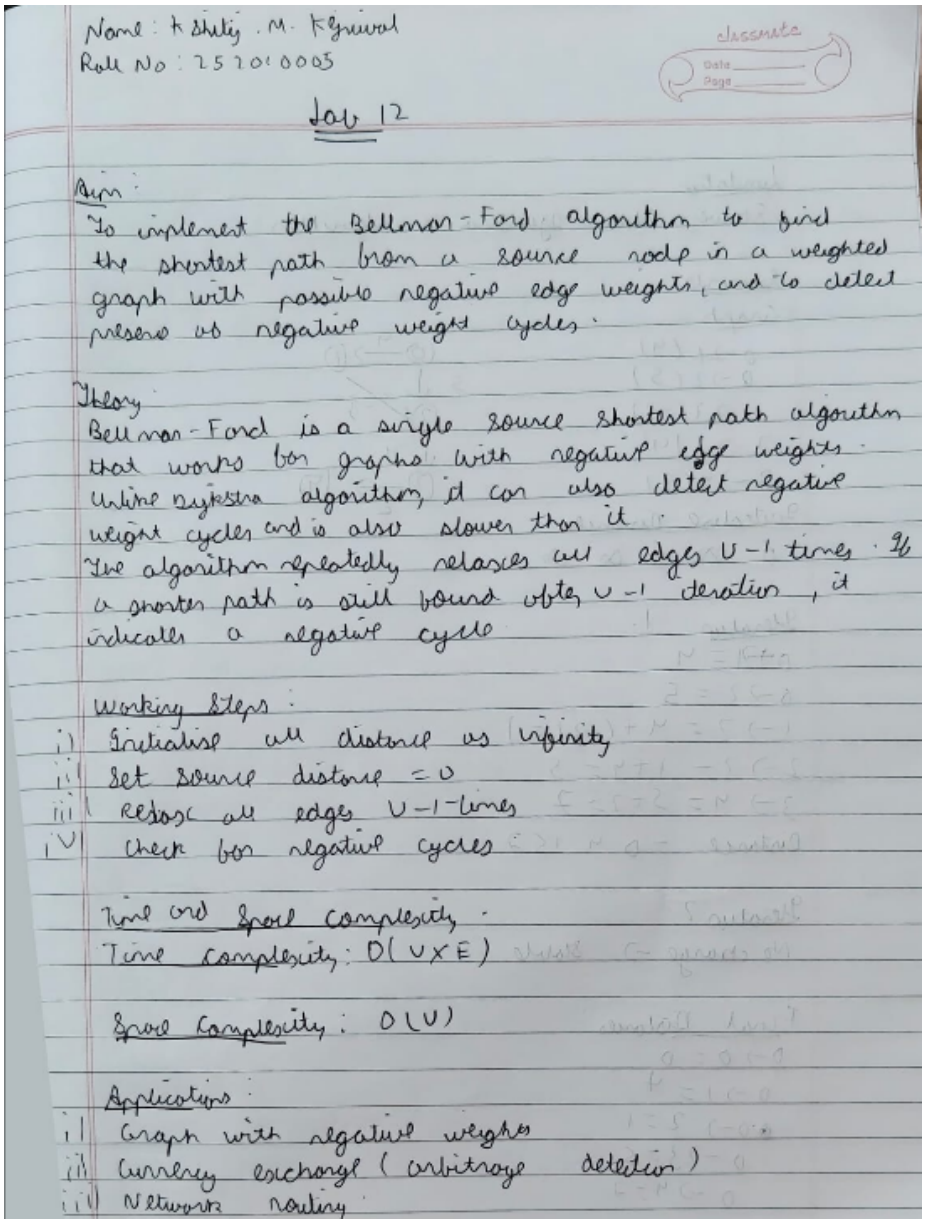
Figure 1: Output

3 Pseudocode:

Algorithm 1 Bellman-Ford Algorithm

```
procedure BellmanFord(G, source)
    Initialize distance array with ∞
    dist[source] ← 0
    for i ← 1 to V − 1 do
        for each edge (u, v, w) in graph do
            if dist[u] + w < dist[v] then
                dist[v] ← dist[u] + w
            end if
        end for
    end for
    for each edge (u, v, w) do
        if dist[u] + w < dist[v] then
            Print "Negative cycle detected"
        end if
    end for
end procedure
```

4 Explanation:



Name : K Shitij . M. K Gnirval
Roll No : 2520100005

Lab 12

Aim :
To implement the Bellman-Ford algorithm to find the shortest path from a source node in a weighted graph with possible negative edge weights, and to detect presence of negative weight cycles.

Theory
Bellman-Ford is a single source shortest path algorithm that works for graphs with negative edge weights. Unlike Dijkstra algorithm, it can also detect negative weight cycles and is also slower than it.
The algorithm repeatedly relaxes all edges V-1 times. If a shorter path is still found after V-1 iteration, it indicates a negative cycle

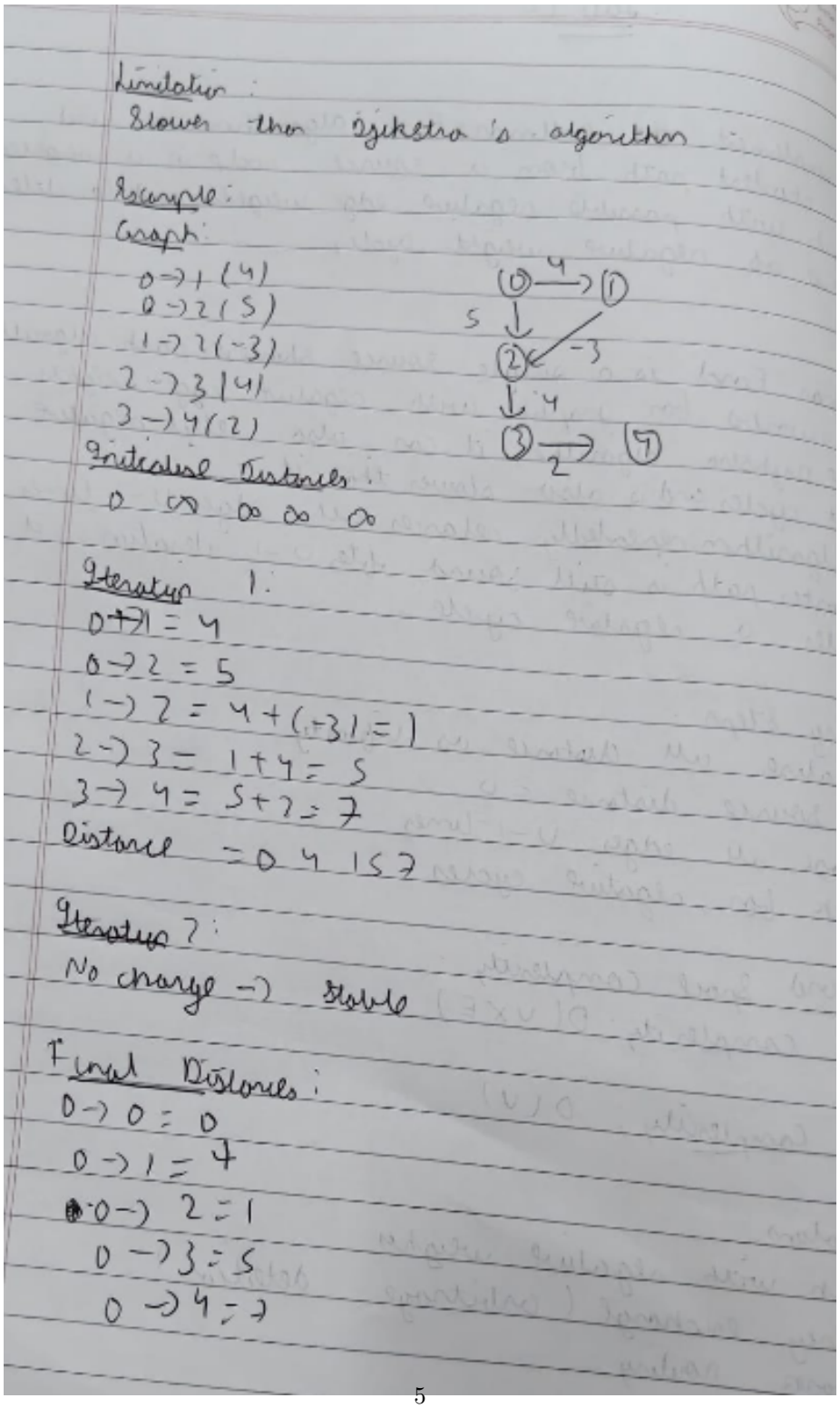
Working Steps :
i) Initialise all distance as infinity
ii) Set source distance = 0
iii) Relax all edges V-1-times
iv) Check for negative cycles

Time and Space complexity :
Time complexity : O(V×E)

Space Complexity : O(V)

Applications :
i) Graph with negative weights
ii) Currency exchange (arbitrage detection)
iii) Network routing

Figure 2: Explanation



Limitation:

Slower than Dijkstra's algorithm

Example:

Graph:

0 -> 1 (4)
0 -> 2 (5)
1 -> 2 (-3)
2 -> 3 (4)
3 -> 4 (2)

Initialise Distances:

0 ∞ ∞ ∞ ∞

Iteration 1:

0 -> 1 = 4
0 -> 2 = 5
1 -> 2 = 4 + (-3) = 1
2 -> 3 = 1 + 4 = 5
3 -> 4 = 5 + 2 = 7

Distance = 0 4 1 5 7

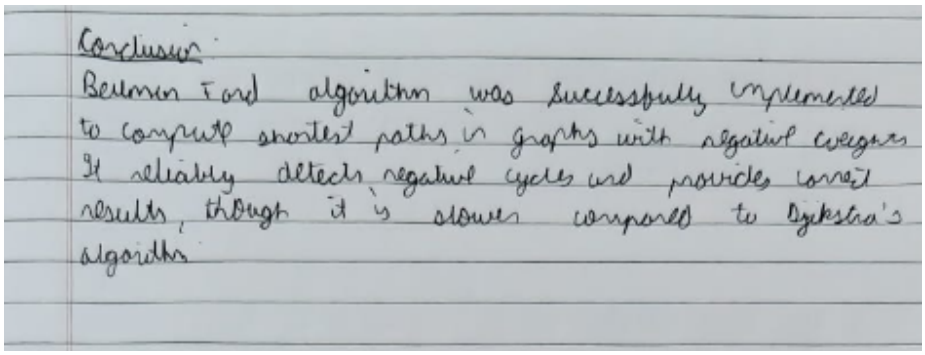
Iteration 2:

No change -> Stable

Final Distances:

0 -> 0 = 0
0 -> 1 = 4
0 -> 2 = 1
0 -> 3 = 5
0 -> 4 = 7

Figure 3: Explanation



Conclusion:

Bellman Ford algorithm was successfully implemented to compute shortest paths in graphs with negative weights. It reliably detects negative cycles and provides correct results, though it is slower compared to Dijkstra's algorithm.

Figure 4: Explanation